

PRACTICAL SESSION-5

TASK 1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
vennela_07@VennelaPC: ~/pr × + v
vennela_07@VennelaPC:~$ mkdir practical5
cd practical5
vennela_07@VennelaPC:~/practical5$ nano task1.c
vennela_07@VennelaPC:~/practical5$ gcc task1.c -o task1
vennela_07@VennelaPC:~/practical5$ ./task1
Consumer received: Data 1
Consumer received: Data 1
Consumer received: Data 1
Consumer received: Data 1
Consumer received: Data 1
Consumer received: Data 1
Consumer received: Data 1
Consumer received: Data 1
Consumer received: Data 1
Consumer received: Data 1
Consumer received: Data 1
Producer: Data sent successfully
Communication time: 0.000039 seconds
```

```
vennela_07@VennelaPC: ~/pr × + v
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <time.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <time.h>

int main() {
    int pipefd[2];
    pid_t pid;
    char buffer[100];
    clock_t start, end;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid > 0) {
        close(pipefd[0]);

        start = clock();

        for (int i = 1; i <= 10; i++) {
            sprintf(buffer, "Data %d", i);
            write(pipefd[1], buffer, strlen(buffer) + 1)
        }

        close(pipefd[1]);
        wait(NULL);

        end = clock();
    }
}
```

GNU nano 7.2

```
        (double)(end - start) / CLOCKS_PER_SEC);
    }
    else {
        close(pipefd[1]);

        for (int i = 1; i <= 10; i++) {
            read(pipefd[0], buffer, sizeof(buffer));
            printf("Consumer received: %s\n", buffer);
        }

        close(pipefd[0]);
    }

    return 0;
}
int main() {
    int pipefd[2];
    pid_t pid;
    char buffer[100];
    clock_t start, end;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid > 0) {
        close(pipefd[0]);

        start = clock();

        for (int i = 1; i <= 10; i++) {
            sprintf(buffer, "Data %d", i);
            write(pipefd[1], buffer, strlen(buffer) + 1);
        }

        close(pipefd[1]);
        wait(NULL);
    }
}
```

```

        end = clock();

        printf("Producer: Data sent successfully\n");
        printf("Communication time: %f seconds\n",
               (double)(end - start) / CLOCKS_PER_SEC);
    }
    else {
        close(pipefd[1]);

        for (int i = 1; i <= 10; i++) {
            read(pipefd[0], buffer, sizeof(buffer));
            printf("Consumer received: %s\n", buffer);
        }

        close(pipefd[0]);
    }

    return 0;
}

```

TASK2: Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```

vennela_07@VennelaPC:~/practical5$ nano task2.c
vennela_07@VennelaPC:~/practical5$ gcc task2.c -o task2
vennela_07@VennelaPC:~/practical5$ ./task2
-rw-r--r-- 1 vennela_07 vennela_07 1068 Sep 18 03:17 task1.c
-rw-r--r-- 1 vennela_07 vennela_07 765 Sep 18 03:24 task2.c

```

```

GNU nano 7.2 task2.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid1 = fork();

    if (pid1 == 0) {
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp");
        exit(1);
    }

    pid2 = fork();

    if (pid2 == 0) {
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);

```

```

        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    wait(NULL);
    wait(NULL);

    return 0;
}
|

```

